

PHP Programming

Due to the length of some of the code lines, this page may not display properly on all browsers or all resolutions.

PHP is a server-side scripting language that can be used on a host of web servers and platforms. I prefer to use it with Apache on either FreeBSD or Linux, but it can even run on Win32 platforms.

What server-side scripting language means is that the script is put into the HTML files that make up a site, but the server processes the script BEFORE it is sent to the client browser. PHP code is not visible if you view the source of a page because the server processes the code and returns only the output. This is easier to code and debug than writing CGI scripts in Perl or C since the HTML form and related code are all in one page and PHP puts any errors on the browser.

Another advantage that PHP offers is the ability to directly connect to relational databases using full featured internal functions. It supports a whole fleet of databases including Oracle, DB2, mSQL and MySQL. I have written this tutorial with an eye to the MySQL database server since that is what this site uses. Most of the information here should be applicable regardless of the backend database.

The Guestbook

To highlight how easy it is to do database backed websites I wrote a simple guestbook using PHP. The code will be examined here to explain how the PHP code works and why I did certain things. One of my goals in writing database backed websites is to keep related pages and functions as simple as possible, but where possible to use a single file to make troubleshooting easier. This might make things more confusing at first, but it will work better in the long term.

The application is very simple. When you go to the page it loads the "Add A Message" form and then connects to the database to get all the guestbook entries. Once it has selected the data it builds the html to display the output and delivers it to the browser.

When someone fills out the form and hits the submit button, the form action reloads the page, adds the data to the database, then selects all the data again including the new entry and builds the html to display the output and delivers it to the browser.

The code I have commented on this page is available in my [File Area](#). In my comments I have used teal to represent comments, red to indicate PHP code and blue to indicate HTML.

Getting Started

The first item of note about PHP is how the server knows whether a bit of text in the file is PHP or HTML. The server is alerted to begin processing PHP code when it encounters a `<?`. It continues to process the text as PHP code until it encounters a `?>`. The wonderful part about PHP is that you can break into and out of PHP processing mode for little snippets of code.

Lets start by looking at the guestbook program as an example. The very first line of code `<?` tells the server to enter PHP mode. The next line `/*` should look familiar to any Perl, C or Java programmers out there as the beginning of a comment. That kind of comment will run until it hits a `*/`

```
<?
/*
PHP Guestbook
Written by Tony Awtrey
```

Anthony Awtrey Consulting

See <http://www.awtrey.com/support/dbeweb/> for more information

- 1.1 - Oct. 20, 1999 - changed the SQL statement that reads data back out of the database to reverse the order putting the newest entries at the top and limiting the total displayed by default to 20. Added the ability to get the complete list by appending the URL with '?complete=1'. Added the code and additional query to count and list the total number of entries and included a link to the complete list.
- 1.0 - Initial release

This is the SQL statement to create the database required for this application.

```
CREATE TABLE guests (
  guest_id
    int(4)
    unsigned
    zerofill
    DEFAULT '0000'
    NOT NULL
    auto_increment,
  guest_name varchar(50),
  guest_email varchar(50),
  guest_time timestamp(14),
  guest_message text,
  PRIMARY KEY (guest_id)
);

*/
```

This next section of code demonstrates a second kind of comment that PHP shares with C++, Perl and Javascript. Any line that starts with `//` is taken as a comment. The next line of actual PHP code tests to see if this page is being loaded as a HTTP POST query. When HTML forms are processed, they can either put data to the server using the GET method or the POST method. GET is the one that puts a `?` after the page followed by a string of key=value pairs. POST methods hide the data being input by listing it is a special environment variable called `HTTP_POST_VARS`. If we are posting, this is the beginning of how we put data into the database.

```
////////////////////////////////////
// This checks to see if we need to add another guestbook entry.
////////////////////////////////////
if (($REQUEST_METHOD=='POST')) {
```

This next section of code removes the dangerous characters from the data from the form. Any input recieved from an unverified source like the Internet should be treated as if it were an attack or crack attempt. To prevent this I remove the redirection characters (`<>`) and the "pipe" character (`|`) and replace them with spaces. PHP provides a complete set of string handling functions, including the `strtr` or "string translate" function.

Another nice thing it does is automatically adds the backslashes to quote characters for you to keep from causing problems sending the string data to the database.

The last item of note is the use of the `$$key` to indirectly refer to a variable. This code basically walks through each key=value pair and temporarily assigns the value to a variable called `$this`. Once the data has been untainted it is assigned back into the original value statement

```
////////////////////////////////////
```

```
// This loop removed "dangerous" characters from the posted data
// and puts backslashes in front of characters that might cause
// problems in the database.
////////////////////////////////////
for(reset($HTTP_POST_VARS);
    $key=key($HTTP_POST_VARS);
    next($HTTP_POST_VARS)) {
    $this = addslashes($HTTP_POST_VARS[$key]);
    $this = strtr($this, ">", " ");
    $this = strtr($this, "<", " ");
    $this = strtr($this, "|", " ");
    $$key = $this;
}
```

Now we need to check that all the fields were filled in completely. If they were filled out we are ready to put the data into the table.

```
////////////////////////////////////
// This will catch if someone is trying to submit a blank
// or incomplete form.
////////////////////////////////////
if ($name && $email && $message ) {
```

First we define the INSERT query we are going to send. PHP automatically puts the data on the form into variables named the same as the form INPUT tags. **\$name**, **\$email** and **\$message** are all created by PHP when the form is posted. These expand out in the definition of the **\$query** variable. Note the **.=** symbol. This concatenates the subsequent **\$query** definitions with the first instead of redefining **\$query**.

The three lines that begin with "mysql" are all it takes to put data into a database. The first line connects to the database host using the supplied username and password, the second line selects the database to have the data updated to and the third line issues the query. the "or die" statements are a primitive error handler that will tell you at least what failed if you did something wrong.

```
////////////////////////////////////
// This is the meat of the query that updates the guests table
////////////////////////////////////
$query = "INSERT INTO guests ";
$query .= "(guest_id, guest_name, ";
$query .= "guest_email, guest_time, guest_message) ";
$query .= " values(0000, '$name', '$email', NULL, '$message')";
mysql_pconnect("db2.pair.com", "tator_w", "password")
    or die("Unable to connect to SQL server");
mysql_select_db("tator_awtrey") or die("Unable to select database");
mysql_query($query) or die("Insert Failed!");
```

If the form was NOT completely filled out a variable called **\$notall** is created and set to "1". If the form was NOT called via the POST method all the preceding code was skipped down to the last bracket. Then the **?>** indicates we are back in HTML land.

```
} else {

    //////////////////////////////////
    // If they didn't include all the required fields set a variable
    // and keep going.
    //////////////////////////////////
    $notall = 1;

}
}
```

```
?>
```

This next section is the beginning of the standard HTML part of the page.

```
<!-- Start Page -->
<HTML>
<HEAD>
<TITLE>Add a Message</TITLE>
</HEAD>

<BODY BGCOLOR="white">

<H1>Add A Message</H1>
```

This next section is a good example of how to mix and match PHP and HTML. The first line is a standard HTML comment. The second line switches to PHP and sets an IF statement to detect if the **\$notall** variable was set to indicate that the form was incomplete. If it was set, then output the next line *which is HTML, not even PHP code*, and continue. If it was not set the HTML is not returned to the browser. How about that?!

```
<!-- Let them know that they have to fill in all the blanks -->
<? if ($notall == 1) { ?>
<P><FONT COLOR="red">Please answer all fields</FONT></P>
<? } ?>
```

This is the section that creates the form on the browser. The things to note are the little snippets of PHP to fill in the fields on the form if the person submitted an incomplete form. This keeps them from having to input ALL the data in again.

```
<!-- The bits of PHP in the form allow the data that was already input
      to be placed back in the form if it is filled out incompletely -->

<FORM METHOD="post" ACTION="guest.php">
<PRE>
Your Name:      <INPUT
                  TYPE="text"
                  NAME="name"
                  SIZE="20"
                  MAXLENGTH="50"
                  VALUE="<? echo $name; ?>">
Your Email:     <INPUT
                  TYPE="text"
                  NAME="email"
                  SIZE="20"
                  MAXLENGTH="50"
                  VALUE="<? echo $email; ?>">

Enter Message:
<TEXTAREA NAME="message" COLS="40" ROWS="8" WRAP="Virtual">
<? echo $message; ?>
</TEXTAREA>

<INPUT TYPE="submit" VALUE="Add">

</PRE>
</FORM>

<HR>
```

Here we have the same two lines that connect to the host and select the database as before. The

query is defined and called.

```
<?

////////////////////////////////////
// This is where we connect to the database for reading.
////////////////////////////////////
mysql_pconnect("db2.pair.com","tator_r","password")
    or die("Unable to connect to SQL server");
mysql_select_db("tator_awtrey") or die("Unable to select database");
```

This next section sends a query to the database to count the total number of records in the table that have data in them. This is an easy way to count records.

```
////////////////////////////////////
// This is where we count the number of entries.
////////////////////////////////////
$query = "SELECT COUNT(*) FROM guests";
$numguests = mysql_query($query) or die("Select Failed!");
$numquest = mysql_fetch_array($numguests);

?>

<!-- This is where we report the total messages. -->
<P>
<A HREF="guest.php?complete=1"><? echo $numquest[0]; ?> people</A> have
left me a message.
</P>
```

This query is set depending on whether the URL is set to retrieve all the records or not.

```
////////////////////////////////////
// This is where we decide to get all the entries or just the last 20.
// This variable is set by just adding a '?complete=1' after the URL.
////////////////////////////////////
if ($complete == 1) {
    $query = "SELECT * FROM guests ORDER BY guest_time DESC";
} else {
    $query = "SELECT * FROM guests ORDER BY guest_time DESC LIMIT 20";
}
$guests = mysql_query($query) or die("Select Failed!");
```

This next section is a loop that runs as long as there are records. The **while** statement returns each record to an array called **\$guest**. It contains all the data selected from the table in array variables named for the field name. Therefore if you want the "guest_name" value, you can get it by referring to **\$guest['guest_name']** This is an incredibly powerful feature since it allows you to change the structure of a database without breaking the application.

```
<?

////////////////////////////////////
// This will loop as long as there are records waiting to be processed.
// Notice the plain HTML inside the while loop structure. PHP is flexible
// enough to allow you to break into and out of the "code" at any point.
////////////////////////////////////
while ($guest = mysql_fetch_array($guests)) {

?>
```

This is the actual beginning of the loop. Most of it is standard HTML with PHP inserted to supply the database fields.

```
<TABLE BORDER="1" WIDTH="500">
<TR><TD>
Name: <? echo $guest['guest_name']; ?>
</TD><TD>
Email: <A HREF="mailto:<? echo $guest['guest_email']; ?>">
      <? echo $guest['guest_email']; ?></A>
</TD><TD>
```

Using the MySQL timestamp datatype in PHP requires a little coding. This next bit takes apart the MySQL data and generates a formatted date. The **substr** function just grabs a substring from a given string. The **mktime** function generates a timestamp that the **date** function uses to generate the date and time in the format specified.

```
<?

////////////////////////////////////
// The database has a timestamp record type that we can use to show the
// date the guestbook was filled out.
////////////////////////////////////
$datefromdb = $guest['guest_time'];
$year = substr($datefromdb,0,4);
$mon  = substr($datefromdb,4,2);
$day  = substr($datefromdb,6,2);
$hour = substr($datefromdb,8,2);
$min  = substr($datefromdb,10,2);
$sec  = substr($datefromdb,12,2);
$orgdate = date("l F dS, Y h:i A",mktime($hour,$min,$sec,$mon,$day,$year));
?>
Date: <? echo $orgdate; ?>
```

The last bit just uses PHP to drop in the variables where needed. The closing **}** ends the while loop and once all the data is output, the page finishes up with standard HTML.

```
</TD></TR>
<TR><TD COLSPAN="3">
<? echo $guest['guest_message']; ?>
</TD></TR>
</TABLE>
<BR>

<? } ?>

</BODY>
</HTML>
```